

Numpy model values

```
import numpy as np
from quantise_fun import quantise
from scipy.signal import firwin, lfilter
import matplotlib.pyplot as plt

numtaps = 16

cutoff_freq = 2000
fs=10000
cutoff=cutoff_freq/(fs/2)

coefficients = firwin(numtaps, cutoff,window='rectangular',pass_zero=True)
# what firwin is doing is that it is first calculate the ideal impulse reponce of what we nee
# then it woudl shift it to make it causal and then to truncate the sinc fucntion it multiply it with the
window
# and then give the coefficients of the discreate signal created that are its values

print(coefficients)

quantise_value=[quantise(i) for i in coefficients]
print("quantised value",quantise_value)

x = np.array([1,2,3,4,5])
y_numpy = np.convolve(x, quantise_value)

print("numpy model values",y_numpy)
```

```
(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog$ python ../filter_design_python/
py
[ 1.47508243e-17  4.40593836e-02  3.21811414e-02 -3.93325061e-02
 -8.18245695e-02  1.47508243e-17  1.90923995e-01  3.53992555e-01
  3.53992555e-01  1.90923995e-01  1.47508243e-17 -8.18245695e-02
 -3.93325061e-02  3.21811414e-02  4.40593836e-02  1.47508243e-17]
quantised value [0, 5, 4, -5, -10, 0, 24, 45, 45, 24, 0, -10, -5, 4, 5, 0]
numpy model values [ 0  5 14 18 12  6 -6 28 157 345 483 467 296  84 -42 -48  6 40
25  0]
```

Verilog Code

```
index.v
//taps = 16
//order =15

module filter (
    input wire clk,
    input wire rstn,
    input wire signed [7:0] sig,
    output wire signed [19:0] samp_op,
    output wire signed [7:0] trunc_op
);

    //assume the sig is q1.7
```

```
wire signed [7:0] coeff [0:15];
reg signed [7:0] signal[0:15];
wire signed [15:0] res[0:15];
```

```
assign coeff[0] = 8'sd0;
assign coeff[1] = 8'sd5;
assign coeff[2] = 8'sd4;
assign coeff[3] = -8'sd5;
assign coeff[4] = -8'sd10;
assign coeff[5] = 8'sd0;
assign coeff[6] = 8'sd24;
assign coeff[7] = 8'sd45;
assign coeff[8] = 8'sd45;
assign coeff[9] = 8'sd24;
assign coeff[10] = 8'sd0;
assign coeff[11] = -8'sd10;
assign coeff[12] = -8'sd5;
assign coeff[13] = 8'sd4;
assign coeff[14] = 8'sd5;
assign coeff[15] = 8'sd0;
```

```
integer i;
always @(posedge clk or negedge rstn) begin
```

```
    if(!rstn) begin
        for(i=0;i<16;i=i+1)begin
            signal[i]<=8'sd0;
        end
    end else begin
        for(i=15;i>0;i=i-1) begin
            signal[i]<=signal[i-1];
        end
        signal[0]<=sig;
    end
```

```
end
```

```
genvar k;
generate
```

```
for(k=0;k<16;k=k+1) begin: gen
    mul ml(
```

```
        .delsig(signal[k]),
        .coeff(coeff[k]),
        .res(res[k])
```

```
    );
```

```
end
```

```
endgenerate
wire signed [19:0] sum;
```

```
assign sum = res[0] + res[1] + res[2] + res[3] +
             res[4] + res[5] + res[6] + res[7] +
             res[8] + res[9] + res[10] + res[11] +
             res[12] + res[13] + res[14] + res[15];
```

```
wire signed [19:0] rounded_sum;
assign rounded_sum = sum + 20'sd64;
```

```
wire signed [12:0] scaled_sum;
assign scaled_sum = rounded_sum >>> 7;
```

```
reg signed [7:0] final_trunc;
```

```
always @(*) begin
```

```
    if (scaled_sum > 13'sd127) begin
        final_trunc = 8'sd127;
```

```
    end else if (scaled_sum < -13'sd128) begin
        final_trunc = -8'sd128;
```

```
    end else begin
        final_trunc = scaled_sum[7:0];
    end
```

```
end
```

```
assign samp_op = sum;
assign trunc_op = final_trunc;
endmodule
```

```
module mul(
```

```
    input signed [7:0] delsig,
    input signed [7:0] coeff,
    output signed [15:0] res
```

```
);
```

```
assign res=coeff*delsig;
```

```
endmodule
```

TESTBENCH CODE

```
`timescale 1ns / 1ps

module tb_index;

    reg clk;
    always #(5.0) clk = ~clk;

    reg rst_n = 1'b0;

    // ---- Shared DUT input signals (reg) ----
    reg rstn;
    reg [7:0] sig;

    // ---- Shared DUT output signals (wire) ----
    wire [19:0] samp_op;
    wire [7:0] trunc_op;
    reg signed [7:0] x_input [0:4];
    // ---- DUT: filter (u_filter) ----
    filter u_filter (
        .clk    ( clk    ),
        .rstn   ( rstn   ),
        .sig    ( sig    ),
        .samp_op ( samp_op ),
        .trunc_op ( trunc_op ));

    initial begin
        $dumpfile("tb_index.vcd");
        $dumpvars(0, tb_index);
    end
    integer i;
    initial begin

        clk=0;
        rstn=0;
        sig=8'sd0;

        x_input[0] = 8'sd1;
        x_input[1] = 8'sd2;
        x_input[2] = 8'sd3;
        x_input[3] = 8'sd4;
        x_input[4] = 8'sd5;

        #15 rstn=1;

        for (i = 0; i < 21; i = i + 1) begin
            @(posedge clk);
            if (i < 5)
                sig <= x_input[i];
        end
    end
endmodule
```

```

else
    sig <= 8'sd0;

#1;
$display("cycle = %d sig = %d samp_op = %d trunc_op = %d", i + 1, sig, $signed(samp_op),
$signed(trunc_op));
end
    #20;
$finish;
end

endmodule

```

```

(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog$ vvp sim
VCD info: dumpfile tb_index.vcd opened for output.
cycle =      1 sig =  1 samp_op =      0 trunc_op =      0
cycle =      2 sig =  2 samp_op =      0 trunc_op =      0
cycle =      3 sig =  3 samp_op =      5 trunc_op =      0
cycle =      4 sig =  4 samp_op =     14 trunc_op =      0
cycle =      5 sig =  5 samp_op =     18 trunc_op =      0
cycle =      6 sig =  0 samp_op =     12 trunc_op =      0
cycle =      7 sig =  0 samp_op =      6 trunc_op =      0
cycle =      8 sig =  0 samp_op =     -6 trunc_op =      0
cycle =      9 sig =  0 samp_op =     28 trunc_op =      0
cycle =     10 sig =  0 samp_op =    157 trunc_op =      1
cycle =     11 sig =  0 samp_op =    345 trunc_op =      3
cycle =     12 sig =  0 samp_op =    483 trunc_op =      4
cycle =     13 sig =  0 samp_op =    467 trunc_op =      4
cycle =     14 sig =  0 samp_op =    296 trunc_op =      2
cycle =     15 sig =  0 samp_op =     84 trunc_op =      1
cycle =     16 sig =  0 samp_op =    -42 trunc_op =      0
cycle =     17 sig =  0 samp_op =    -48 trunc_op =      0
cycle =     18 sig =  0 samp_op =      6 trunc_op =      0
cycle =     19 sig =  0 samp_op =     40 trunc_op =      0
cycle =     20 sig =  0 samp_op =     25 trunc_op =      0
cycle =     21 sig =  0 samp_op =      0 trunc_op =      0
tb_index.v:59: $finish called at 236000 (1ps)

```

the numpy model and the created rtl code filter output matches

IMPULSE RESPONSE OF PYTHON AND VERILOG IMPLEMENTATION

```

(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog$ python ../filter_desi
py
[ 1.47508243e-17  4.40593836e-02  3.21811414e-02 -3.93325061e-02
 -8.18245695e-02  1.47508243e-17  1.90923995e-01  3.53992555e-01
  3.53992555e-01  1.90923995e-01  1.47508243e-17 -8.18245695e-02
 -3.93325061e-02  3.21811414e-02  4.40593836e-02  1.47508243e-17]
quantised value [0, 5, 4, -5, -10, 0, 24, 45, 45, 24, 0, -10, -5, 4, 5, 0]
numpy model values [ 0  5  4 -5 -10  0 24 45 45 24  0 -10 -5  4  5  0]

```

```
(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog$ vvp sim
VCD info: dumpfile tb_index.vcd opened for output.
cycle =      1 sig =  1 samp_op =    0 trunc_op =    0
cycle =      2 sig =  0 samp_op =    0 trunc_op =    0
cycle =      3 sig =  0 samp_op =    5 trunc_op =    0
cycle =      4 sig =  0 samp_op =    4 trunc_op =    0
cycle =      5 sig =  0 samp_op =   -5 trunc_op =    0
cycle =      6 sig =  0 samp_op =  -10 trunc_op =    0
cycle =      7 sig =  0 samp_op =    0 trunc_op =    0
cycle =      8 sig =  0 samp_op =   24 trunc_op =    0
cycle =      9 sig =  0 samp_op =   45 trunc_op =    0
cycle =     10 sig =  0 samp_op =   45 trunc_op =    0
cycle =     11 sig =  0 samp_op =   24 trunc_op =    0
cycle =     12 sig =  0 samp_op =    0 trunc_op =    0
cycle =     13 sig =  0 samp_op =  -10 trunc_op =    0
cycle =     14 sig =  0 samp_op =   -5 trunc_op =    0
cycle =     15 sig =  0 samp_op =    4 trunc_op =    0
cycle =     16 sig =  0 samp_op =    5 trunc_op =    0
cycle =     17 sig =  0 samp_op =    0 trunc_op =    0
cycle =     18 sig =  0 samp_op =    0 trunc_op =    0
cycle =     19 sig =  0 samp_op =    0 trunc_op =    0
cycle =     20 sig =  0 samp_op =    0 trunc_op =    0
cycle =     21 sig =  0 samp_op =    0 trunc_op =    0
tb_index.v:59: $finish called at 236000 (1ps)
```